

OpenStore

GitHub AI Explorer — Documentación Técnica y de Producto

Versión	1.0 — septiembre de 2026
Autor / Operador	Jose Quevedo
Repositorio	https://github.com/Josequevedov08/OpenStore
App en vivo	https://app-repositorio-github-one.vercel.app
Stack	React + Vite + Tailwind (frontend) · FastAPI (backend)
Hosting	Vercel (frontend) · Render (backend, capa gratuita)
Licencia	Ver LICENSE en el repositorio

1. Qué es OpenStore

OpenStore es un motor de búsqueda de repositorios públicos de GitHub que reemplaza un README técnico por una ficha comercial escrita por IA, en lenguaje sencillo. Quien visita la app escribe lo que necesita en palabras simples ("un CRM", "una app de lista de tareas", la URL de un repositorio, o un usuario de GitHub), y la app devuelve un conjunto de tarjetas estilo tienda de aplicaciones: un título llamativo, una ficha corta explicando qué hace el proyecto y por qué importa, sus características clave, qué se necesita para correrlo y, cuando el README lo permite, un instalador de un clic para la propia máquina de quien lo pida.

No hay registro, no hay cuenta, y no hay muro de pago. El proyecto corre por completo sobre infraestructura de capa gratuita (la API pública de GitHub, un proveedor de IA de capa gratuita, hosting gratuito en Vercel y Render), y por eso varias de las decisiones de ingeniería documentadas aquí existen específicamente para que un despliegue gratuito se comporte de forma confiable bajo límites de uso reales y compartidos.

2. Arquitectura

El sistema es una arquitectura clásica de dos capas: un frontend estático de página única y un backend API sin estado, sin base de datos. Todo el estado que necesita persistir vive o bien en el propio localStorage del navegador (por visitante, nunca se envía a ningún lado) o en estructuras de vida corta en memoria en el backend (caché, limitador de tasa, contadores de analítica) que se reinician cada vez que el proceso se reinicia.

2.1 Frontend

React 19 + Vite, con estilos de Tailwind CSS v4 (configuración basada en CSS, sin tailwind.config.js). Desplegado como build estático en Vercel. Todo el tematizado — claro/oscuro — se controla con propiedades personalizadas de CSS definidas una sola vez en index.css y usadas en todas partes mediante la sintaxis de valores arbitrarios de Tailwind (p. ej. bg-[var(--surface)]), así que ningún componente tiene un color fijo en su código.

2.2 Backend

Python 3 + FastAPI, desplegado en la capa gratuita de Render (el servicio se duerme tras inactividad — la primera petición después de un período inactivo puede tardar 30-50 segundos solo en despertarlo; el frontend hace ping a /health proactivamente al cargar para adelantar esto). El backend orquesta cuatro fases por búsqueda: (1) encontrar repositorios candidatos vía la API de Búsqueda de GitHub, (2) obtener los metadatos y el README crudo de cada repositorio en paralelo, (3) enviar cada README a un modelo de IA para producir una ficha comercial estructurada, y (4) ensamblar la respuesta final, sanitizando todo lo que después se use para construir un script instalador.

2.3 Flujo de datos de una búsqueda

- El navegador envía el texto de la búsqueda, el idioma seleccionado, la página pedida y el orden a POST /api/buscar-soluciones.
- El backend revisa un limitador de tasa para la IP de quien llama, y luego una caché en memoria indexada por (consulta, idioma, página, orden).
- Si no hay caché, consulta la API de Búsqueda de GitHub por hasta 12 repositorios por página, y descarga el README de cada uno en paralelo.

- Cada README se envía al proveedor de IA configurado con una instrucción estricta: traducir todo al idioma pedido y devolver una forma JSON específica (título, ficha, características, requisitos, tecnologías, gestor de paquetes, comando de arranque).
- Las llamadas a la IA corren con un límite de concurrencia y un tiempo máximo de espera por llamada, así que una sola llamada lenta o colgada nunca bloquea toda la búsqueda; cualquier repo cuya llamada a la IA falle cae a un respaldo honesto y claramente etiquetado, construido con los metadatos crudos de GitHub.
- La respuesta se guarda en caché (solo si al menos una ficha vino de una llamada real a la IA) y se devuelve al navegador.

3. Características principales

- **Búsqueda en lenguaje natural** sobre todos los repositorios públicos de GitHub, además de URLs de repos, URLs de perfiles de GitHub, y el atajo "@usuario" para explorar los repositorios de una persona u organización.
- **Ficha comercial por repositorio, escrita por IA**: un gancho, qué es, cómo funciona y un llamado a la acción — en vez de documentación técnica cruda.
- **Bilingüe por diseño**: inglés por defecto, español como opción. La IA traduce automáticamente desde el idioma original del README, cualquiera que sea.
- **Ordenar por más estrellas o por actualizados recientemente**, con un piso de calidad (un mínimo de estrellas) aplicado automáticamente a "actualizados" para que no salgan a la luz repos oscuros y con apenas estrellas que solo coinciden con el texto por casualidad.
- **"Cargar más" en vez de páginas numeradas**: los resultados se acumulan en una sola pantalla continua.
- **El Instalador Inteligente**: un script descargable (.bat para Windows, un .sh que detecta el sistema operativo para macOS/Linux) que revisa que Git y el runtime correcto estén instalados, ofrece instalar las herramientas faltantes con el gestor de paquetes propio del sistema operativo, clona el repositorio, instala sus dependencias y lo ejecuta — todo con un doble clic. Ver la Sección 5 para cómo se mantiene esto seguro.

4. Características avanzadas

- **Tema claro/oscuro real**, guardado por navegador y aplicado antes del primer dibujo de la página (sin parpadeo del tema incorrecto).
- **Favoritos**: marca cualquier resultado y navégalos en una vista dedicada.
- **Comparar**: elige hasta tres repositorios y vélos lado a lado — estrellas, forks, issues abiertos, lenguaje, licencia y tecnologías detectadas.
- **Filtros del lado del cliente**: acota el conjunto de resultados actual por lenguaje, licencia, o "tiene instalador".
- **Historial de instalación**: cada descarga exitosa de un instalador queda registrada localmente con su título, repositorio, plataforma y fecha.
- **Accesibilidad**: un contorno de foco de teclado visible en toda la interfaz.
- **Instalable como PWA**: un manifiesto de app web y un service worker mínimo permiten instalar OpenStore como una app nativa en escritorio o móvil. Solo el cascarón de la app funciona sin conexión — buscar siempre necesita conexión en vivo a GitHub y al proveedor de IA.
- Todo lo anterior (tema, favoritos, comparaciones, búsquedas recientes, historial de instalación) vive exclusivamente en el propio navegador de quien visita (localStorage) — nunca se envía al backend, nunca está atado a una identidad, porque no existe ninguna a la cual atarlo.

5. Diseño de seguridad

La parte más delicada de este sistema es el Instalador Inteligente: convierte texto escrito por un tercero (el README de un repositorio) y leído por un modelo de IA en un script que una persona real hará doble clic para correr en su propia máquina. Ese proceso se trata como completamente no confiable de principio a fin.

5.1 Sanitización de comandos

A la IA nunca se le permite escribir un comando de instalación arbitrario como texto libre. Solo puede elegir un gestor de paquetes de un conjunto cerrado y fijo en el código (npm, yarn, pnpm, pip, poetry, cargo, go, bundler, composer, dotnet, docker, o "none") — cada uno mapeado, del lado del servidor, a un comando literal y ya escrito. El campo separado "comando de arranque" que la IA extrae del README (p. ej. "npm run dev") pasa por un validador estricto de lista blanca/negra antes de poder llegar a un script generado: rechaza el encadenado de comandos (&&, ;, |), redirecciones, herramientas de descarga de red (curl, wget), escalado de privilegios (sudo), PowerShell codificado/ofuscado, y cualquier cosa que no esté construida a partir de una lista corta de binarios conocidos como seguros. Esto está cubierto por una batería de pruebas automatizadas (16+ casos) con payloads deliberadamente adversarios (p. ej. "... && rm -rf /").

5.2 Lo que la sanitización no puede hacer

Ninguna cantidad de sanitización de comandos puede auditar el código fuente real de un repositorio de terceros. El instalador solo controla cómo se descarga y arranca el proyecto — una vez que corre, es exactamente tan confiable como el código de su propio autor. Esta limitación se muestra explícitamente a quien visita en las Preguntas Frecuentes, los Términos y el manual visual, repitiendo en todos lados el mismo consejo de una línea: instala solo repositorios de autores en los que confíes.

5.3 Protección contra abuso y agotamiento de cuota

- Límite de tasa por IP en el endpoint de búsqueda (configurable, por defecto 10 peticiones / 5 minutos) protege la cuota gratuita compartida de IA y GitHub de ser agotada por una sola persona o script.
- Una caché de búsquedas en memoria de 15 minutos hace que una consulta idéntica se sirva al instante sin gastar nada de cuota de IA o GitHub.
- Se puede configurar un segundo proveedor de IA como respaldo automático: si el proveedor principal falla para un repositorio específico (cuota, timeout, respuesta mal formada), ese repositorio puntual se reintenta con el respaldo antes de rendirse y mostrar un respaldo honesto de "análisis pendiente".
- Un tiempo máximo de espera fijo por cada llamada a la IA evita que una sola llamada colgada bloquee toda una búsqueda.

5.4 Panel de administración

Un endpoint de solo lectura GET /api/admin/stats expone contadores operativos agregados (búsquedas totales, porcentaje de aciertos de caché, proporción de fichas procesadas por IA frente a las de respaldo, los términos de búsqueda más comunes, peticiones bloqueadas por rate limit). Está deshabilitado por defecto (devuelve 503) y solo se activa una vez que quien opera el servicio configura una variable de entorno ADMIN_TOKEN; cada petición se verifica contra ella con una comparación de tiempo constante. Un pequeño panel independiente en /admin.html lee este endpoint — no está enlazado desde la app principal y requiere el token para ver cualquier cosa.

6. Privacidad y manejo de datos

OpenStore no tiene cuentas de usuario y no sabe quién es quien la visita. No hay ningún servicio de analítica de terceros (nada de Google Analytics, Meta Pixel) ni cookies de rastreo.

- El texto de una búsqueda se envía a la API pública de Búsqueda de GitHub y al proveedor de IA configurado, bajo los propios términos de privacidad de cada uno.
- El backend guarda únicamente contadores agregados en memoria para planificar capacidad (ver 5.4) — se borran cada vez que el proceso se reinicia, nunca se escriben a disco, nunca están atados a una IP o una persona.
- Todo lo demás que necesita persistir — tema, favoritos, búsquedas recientes, comparaciones, historial de instalación — vive exclusivamente en el propio navegador de quien visita (localStorage) y nunca se envía al backend.
- Una integración opcional, actualmente inactiva, puede registrar los resultados generados por IA en una hoja de Google privada para el registro propio de quien opera el servicio; solo se activa si esa persona configura manualmente credenciales para ello, y nunca guarda nada que identifique a quien visita.

El texto completo y vigente vive en la propia app en /privacidad.html (Política de Privacidad), /terminos.html (Términos y Condiciones) y /faq.html (Preguntas Frecuentes) — esta sección los resume; esas páginas son la fuente de verdad y se actualizan cada vez que el producto cambia.

7. Referencia de configuración

Toda la configuración del backend se hace vía variables de entorno (ver backend/.env.example en el repositorio).

- **GITHUB_TOKEN** — muy recomendado: 60 peticiones/hora sin él frente a 5,000/hora con un token de solo lectura; una sola búsqueda ya usa cerca de 13 peticiones.
- **AI_PROVIDER / AI_API_KEY / AI_MODEL** — qué servicio de IA usar (gemini, openai, groq, o anthropic) y su modelo. Necesario para las fichas escritas por IA; sin una clave, toda ficha cae a un respaldo con solo metadatos.
- **AI_FALLBACK_PROVIDER / AI_FALLBACK_API_KEY / AI_FALLBACK_MODEL** — segundo proveedor opcional que se reintenta por repositorio si el principal falla.
- **AI_CONCURRENCY / AI_CALL_TIMEOUT_SECONDS** — cuántas llamadas a la IA corren en paralelo por búsqueda, y la espera máxima por llamada antes de caer al respaldo.
- **SEARCH_CACHE_TTL_SECONDS** — cuánto tiempo se sirve desde caché una búsqueda idéntica (por defecto 900s / 15 min).
- **RATE_LIMIT_MAX_PETICIONES / RATE_LIMIT_VENTANA_SEGUNDOS** — límite de tasa de búsqueda por IP (por defecto 10 peticiones / 300s).
- **ADMIN_TOKEN** — habilita el endpoint de estadísticas de administración (Sección 5.4); sin configurar por defecto.
- **CORS_ORIGINS** — lista separada por comas de orígenes de frontend permitidos.
- Frontend: **VITE_API_URL** — la URL base del backend. Vite la incrusta en tiempo de compilación, no en tiempo de ejecución; cambiarla requiere un nuevo despliegue, y pegar algo que no sea una URL simple (p. ej. un enlace en formato Markdown por accidente) rompe cada petición de una forma fácil de confundir con una caída del servidor (ver Sección 9).

8. Pruebas e integración continua

La lógica más crítica en términos de seguridad — la sanitización de comandos del instalador — tiene cobertura de pruebas automatizadas: se confirma que los comandos válidos se aceptan, y que una batería de entradas adversarias (encadenado de comandos, one-liners destructivos, PowerShell codificado, descargadores de red, escalado de privilegios, cadenas demasiado largas) se rechaza. También están cubiertos los estados de autenticación del endpoint de administración (token faltante, token incorrecto, endpoint deshabilitado, token correcto) y los contadores de analítica.

Un workflow de GitHub Actions corre la batería de pytest del backend y un build de producción del frontend en cada push y pull request contra la rama main, así que un cambio que rompa cualquiera de los dos se detecta antes de llegar a producción.

9. Limitaciones conocidas y lecciones aprendidas

- **Cuotas gratuitas compartidas.** Todo corre sobre capas gratuitas (API de GitHub, proveedor de IA, Render, Vercel). El uso simultáneo intenso de muchas personas puede ocasionalmente agotar una cuota compartida; la app se degrada de forma honesta (un respaldo de "análisis pendiente" claramente etiquetado) en vez de fallar en silencio o mostrar datos inventados.
- **Arranques en frío.** La capa gratuita de Render duerme el backend tras inactividad; la primera petición después de un período inactivo puede tardar 30-50 segundos. El frontend lo despierta proactivamente al cargar la página, antes de que quien visita siquiera busque algo.
- **El instalador no puede auditar código fuente de terceros.** La sanitización protege el comando que arranca un proyecto, no la lógica propia del proyecto una vez que corre (ver Sección 5.2).
- **Una variable de entorno de tiempo de compilación es fácil de corromper en silencio.** Un enlace en formato Markdown pegado en VITE_API_URL en vez de una URL simple rompió una vez cada petición en producción, con un mensaje de error lo bastante genérico como para parecer una caída del servidor en vez de un error de configuración. La solución fue agregar una verificación explícita y visible del formato de la URL que falla ruidosamente en la consola del navegador al arrancar.
- **Las variables de entorno renombradas pueden dejar valores viejos de tipo incorrecto** en el panel del hosting. Una variable renombrada de un nombre de tipo flotante a uno solo de enteros dejó un valor decimal esperando en el panel, lo cual habría tumbado el proceso en el siguiente reinicio. La solución fue un parseo defensivo que registra una advertencia y cae a un valor por defecto en vez de lanzar una excepción.
- **Un parámetro de URL nunca debe decidir a dónde se envía un secreto.** El panel de administración (Sección 5.4) originalmente leía la URL de su backend desde su propio parámetro "?api=" sin validarlo, y luego enviaba el token de administración ahí y persistía ese host en localStorage. Un enlace construido como "<dominio-real>/admin.html?api=https://atacante.example" llevaba un dominio genuino y un candado válido — nada en la barra de direcciones habría alertado a quien lo abriera — y en el momento en que esa persona enviaba su token, el propio código del lado del cliente de la página se lo entregaba directo al atacante, sin necesitar explotar el backend. La solución fue una lista blanca estricta (el origen propio del backend de producción, o localhost solo para desarrollo local) verificada tanto contra el parámetro de la URL como contra cualquier valor ya guardado en localStorage; cualquier otro valor se ignora por completo y nunca se persiste. Los endpoints de administración o que manejan credenciales deben validar los orígenes de destino contra una lista blanca fija — un parámetro de URL no debe decidir el destino de un secreto, porque incluso un enlace del mismo dominio puede convertir a la propia página en el mecanismo de entrega de un phishing, y la persistencia en localStorage extiende la exposición más allá del primer clic. Ver SECURITY.md para el detalle completo.

10. Primeros pasos (desarrollo local)

- Backend: crea backend/.env a partir de backend/.env.example, rellena al menos AI_API_KEY (e idealmente GITHUB_TOKEN), y luego corre "pip install -r requirements.txt" y "uvicorn main:app --reload" desde la carpeta backend/.
- Frontend: crea frontend/.env a partir de frontend/.env.example apuntando VITE_API_URL a tu backend local, y luego corre "npm install" y "npm run dev" desde la carpeta frontend/.
- Pruebas: corre "pytest" desde la carpeta backend/.

11. Enlaces y contacto

- Repositorio: <https://github.com/Josequevedov08/OpenStore>
- App en vivo: <https://app-repositorio-github-one.vercel.app>
- Manual visual, Preguntas Frecuentes, Términos y Condiciones, Política de Privacidad: enlazados desde el pie de página de la app.
- Preguntas sobre el proyecto o el manejo de sus datos: contacta a quien opera el servicio vía GitHub (github.com/Josequevedov08).